

Aquarium Animal Census — technical memo

Kanagavel A K · RAP pre-hackathon screening · September 2026 · github.com/KanagavelAK/aquarium-rt detr

1. Domain and dataset

Problem. Count and rank the animals visible in a public-aquarium camera frame, so an exhibit can be censused from the cameras already on the glass: "how many sharks are in the tank", "which animal is most common", "are the stingrays out". Aquariums do headcounts by hand; a detector that can say *when it cannot count* is more useful than one that always gives a number.

Classes. fish, jellyfish, penguin, puffin, shark, starfish, stingray. None is a COCO class (COCO has *bird*, not penguin or puffin, and no aquatic animal). Three pairs are visually confusable and drive the reasoning design: shark / stingray (grey, seen from above), penguin / puffin (black-and-white birds), and fish / everything small.

Source. Roboflow *Aquarium Combined* via its Kaggle mirror [slavkoprytula/aquarium-data-cots](https://www.kaggle.com/slavkoprytula/aquarium-data-cots), licence **CC BY 4.0**: 638 photographs taken at the Henry Doorly Zoo (Omaha) and the National Aquarium (Baltimore), 4,821 boxes. Chosen because it is real through-the-glass footage with genuine crowding, not stock photography; because every class is already labelled, so no relabelling was needed; and because the imbalance is honest: fish are 55 percent of boxes, starfish 2.4 percent, which the evaluation has to confront rather than hide. `scripts/prepare_data.py` reads the YOLO labels, maps class ids by name, and drops byte-identical duplicates.

2. Split strategy

Image-level, `md5(file_stem)` bucketed 15 / 15 / 70 into test / val / train, ignoring Roboflow's own undocumented folders. Because the assignment is a pure function of the filename (with Roboflow's `.rf.<hash>` suffix removed), re-running or adding data can never move an image from train into test, and the same rule picked the repo's `samples/`. Duplicates are removed by content hash first, so no frame can sit on both sides. 447 images landed in train / 93 val / 98 test; per-class counts are in `artifacts/split_stats.json`. Balance is preserved by chance rather than by stratification — with 638 images, stratifying seven classes at the image level would have meant hand-picking, which is a worse property than a slightly uneven test set.

3. Evaluation and what it means

Held-out test split, 98 images, 787 boxes, Ultralytics `val`, IoU 0.5 for mAP50.

class	test mAP50	mAP50-95	P	R
fish	0.66	0.35	0.61	0.68
jellyfish	0.82	0.49	0.79	0.81
penguin	0.63	0.23	0.61	0.58
puffin	0.50	0.22	0.49	0.60
shark	0.64	0.33	0.61	0.69
starfish	0.45	0.34	0.88	0.43
stingray	0.44	0.29	0.45	0.46
all	0.59	0.32	0.64	0.61

What they tell. Validation mAP50 0.61 vs test 0.59: a 0.014 gap, so selecting `best.pt` on val did not overfit and the split is clean. Recall is the number the census depends on (a missed animal is an undercount the guardrail cannot see): jellyfish 0.81, fish 0.68, shark 0.69, penguin 0.58, starfish 0.43. The confusion matrix (`metrics.json`, `test.confusion`) names the confusions the identity-conflict rule exists for: shark→fish 7, shark↔stingray 5. It also shows the model is **under-trained**: val mAP50 0.13 at epoch 40, 0.46 at 60, 0.54 at 70, 0.61 at 80, best epoch = last.

What they don't. The test split shares photographers, tanks and lighting with training; the reviewer's hidden set will not. mAP ignores what a census must know: at the API's 0.25 threshold the model emits 363 extra fish and 592 extra penguin boxes across the split, almost all below 0.45 and many duplicates of one animal (RT-DETR has no NMS). `failure_cases.py` counts those separately; the reasoning layer collapses duplicates (IoU ≥ 0.7) and asserts nothing from boxes under 0.45.

4. Five failure cases

`failure_cases.py` matches predictions to ground truth (IoU 0.5) on all 98 test images, measures every miss and checks every false positive for being a duplicate query. 90 images have an error; annotated top-8 in `artifacts/failures/`.

1. IMG_2306, distant penguin row: 149 penguin boxes for 12 penguins. The ledge is ~15 px tall at 640; 147 of the 149 boxes score below 0.45 and 62 sit on top of another box (IoU ≥ 0.5). One penguin at 0.05 % of the image is missed (sharp: Laplacian var 2,519). Root cause: scale. The decoder cannot resolve individuals in the row and sprays low-confidence queries. This image set the 0.45 assertion floor; at 0.25 the raw count reads 160.

2. IMG_8421, reef seen from above: 26 fish missed, 77 extra boxes. Missed fish have median area 0.05 % of the image (~14 px), are sharp (Laplacian var 240–9,200) and unoccluded (max overlap 0.05). Root cause: scale at 640 px on a 1024-px photo. The same

image holds three genuine errors: the one shark (0.16 % of the image) predicted *fish* at 0.45, a blue coral predicted *fish* at 0.44, and a purple table edge predicted *stingray* at 0.26. Fix: 800–960 px inference or tiling.

3. IMG_8509, night tank: 46 "false positive" fish, zero duplicates. Every one is a real small fish; the annotators boxed only the four sharks. The metric calls it wrong because the labels are incomplete, not because the model is. Confidence 0.25–0.63, so these count at the API threshold.

4. Stingray → jellyfish, 6 of 41 test stingrays. Roboflow users have documented exactly one mislabelled image in this dataset: seven jellyfish annotated as stingray. The count and the direction match. The most likely reading is that the model is right and the label is wrong; labels are kept as published, so stingray recall (0.46) is understated by about 0.15.

5. Starfish, precision 0.88, recall 0.43. Eight of 18 found, eight missed, two confused (puffin, stingray). Starfish are 2.4 % of training boxes (116) and the run had not converged (best epoch = last). Root cause: class imbalance plus training budget; nothing in the measurements points at blur, light or occlusion.

Pattern. Scale is the dominant genuine cause (cases 1, 2, 5's smaller misses); the rest is the labelling convention (3, 4). Genuine class confusions are rare and specific: shark→fish at small scale, shark↔stingray from above.

5. Reasoning layer (Part B)

One hand-written module, no framework: `route()` → `detector` → `build_scene()` → `guardrail()` → `compose()`.

Route. Deterministic rules over a closed vocabulary sort the question into eight kinds (count, count of kinds, ranking, comparison, presence, summary, not about the image, outside the detector's scope) and extract the classes it names (synonyms folded: *sea star*, *rays*; *jellyfish* never matches *fish*). The last two kinds skip the detector: "capital of France" is not about the image; "what species of shark", "is the penguin healthy", "any baby penguins", "how many people" are about the image but ask for what the detector cannot measure. Rules are faster than a model call and cannot hallucinate a route.

Facts. Duplicate same-class boxes ($\text{IoU} \geq 0.70$) are collapsed, since RT-DETR has no NMS and a second query on one animal must not count twice. Each class gets a count, a crowding score (fraction of its boxes overlapping another of the same class, $\text{IoU} \geq 0.30$) and any identity conflicts (a box of another class on the same pixels, $\text{IoU} \geq 0.50$).

Guardrail, before the language model. Refuse when nothing is detected or every box is below 0.45. Refuse a count when crowding exceeds 34 %, when fewer than half the boxes clear 0.45, or when the class is in a conflict. Refuse a ranking when the top two counts are within 15 % (or one box) of each other. Refuse presence when the only evidence is below the floor. Absence is an answer, not a refusal: "no penguins above 0.25".

Insufficient	information,	worked	example
(tests/test_reasoning.py::test_fish_school_forces_insufficient_information). Fourteen fish boxes at 0.60–0.70: three swimming alone and a school of eleven, each 220 px wide and 90 px apart, so every one overlaps a neighbour. <i>How many fish are in this image?</i> is routed as a count of fish, the detector runs, and the guardrail returns: "I do not have enough information to answer that confidently. 11 of the 14 fish boxes overlap another fish box (crowding 79%), so individual animals in that group cannot be separated and the count may be off in either direction." The same scene answers <i>are there any fish?</i> with "Yes, I detect 14 fish", because presence needs only one confident box.			

Compose. After the guard passes, one direct Messages API call phrases the facts; with no key, templates do, and `answer_source` says which.

6. Reproducibility

notebooks/kaggle_aquarium_rtdetr.ipynb, one Save & Run All on Kaggle (GPU T4 x2, Internet on, no datasets attached): Ultralytics 8.3.40, torch 2.10.0+cu128, Python 3.12, rtdetr-1.pt COCO-pretrained, AdamW lr0 1e-4, batch 16 (8 per GPU, DDP), 640 px, AMP, RAM cache, seed 0, 80-epoch budget with patience 25 → all 80 epochs in 0.50 h (1,805 s) on 2 × T4. artifacts/training_receipt.json records hardware, arguments and wall clock. Dockerfile included (CPU); `space_app.py` serves the demo page and the API in one process (Swagger at /docs).

7. What changed along the way

- 1. Domain: hard-hat compliance → aquarium census.** The first build detected `helmet` / `head` / `person` on construction footage (github.com/KanagavelAK/ppe-rtdetr, complete and deployed). Its dataset labels a person on ~3 percent of workers, so the "which head belongs to which person" reasoning rested on a class the model could not learn (person recall 0.03), and the domain is the default answer to this brief. The census asks the reasoning layer a harder question — is this count trustworthy — with conditions that can be measured on the boxes.
- 2. Kaggle P100 → 2 × T4 DDP:** the current torch build has no `sm_60` kernels; the notebook detects and works around it.
- 3. Ray Tune callback disabled:** Ultralytics auto-registers it and Kaggle's newer ray crashes it after epoch 1.
- 4. Budget:** 80 epochs was chosen for the hour available; the curve says 160 would have been right. Reported as a limit, not hidden.